MINI PROJECT - HOUSE PRICE PREDICTION USING LINEAR REGRESSION

AIM:

To predict house prices from property features such as area, number of rooms, age, distance from the city and parking by implementing a **Linear Regression** model, to evaluate its performance using error metrics, and to deploy it as a login-protected Flask web application.

ALGORITHM:

1. Data Collection: Obtain a housing dataset with property details and sale price (e.g., a Kaggle house price dataset or a local real-estate listing dataset).
2. Data Preprocessing: Handle missing values using the median and remove duplicate records.
3. Feature Selection: Use Area, Bedrooms, Bathrooms, Age, Distance_City and Garage as inputs and Price (in lakhs) as the target.
4. Train-Test Split: Divide the data randomly into training and testing sets (80%-20%).
5. Model Training: Fit a Linear Regression model using the training data.
6. Prediction: Use the trained model to predict house prices on the test set.
7. Evaluation: Assess the model using Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE) and R^2 score.
8. Visualization: Plot actual vs predicted prices to observe prediction accuracy.
9. Deployment: Save the model with joblib and load it in a Flask application with Register, Login and Dashboard pages.
10. Prediction on the Web: Accept property details through a form and display the predicted price in lakhs.

CODE:

train_model.py

```
import os
import json
import joblib
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

CSV_PATH = "house_data.csv"   # columns: Area, Bedrooms, Bathrooms, Age, Distance_City,
    Garage, Price

# ---------- 1. Data Collection ----------
def load_data():
    if os.path.exists(CSV_PATH):
        return pd.read_csv(CSV_PATH)
    # Simulated housing data (same schema as a real dataset) used when no CSV is present
    rng = np.random.default_rng(11)
    n = 1500
    area = rng.uniform(500, 3500, n).round(0)                       # square feet
    bedrooms = np.clip(np.round(area / 750 + rng.normal(0, 0.6, n)), 1, 5).astype(int)
    bathrooms = np.clip(bedrooms - rng.integers(0, 2, n), 1, 4)
```

```
    age = rng.integers(0, 31, n)                                    # years
    distance = rng.uniform(1, 30, n).round(1)                       # km from city
        centre
    garage = rng.integers(0, 2, n)
    price = (10 + 0.045 * area + 4 * bedrooms + 3 * bathrooms - 0.6 * age
             - 1.2 * distance + 5 * garage + rng.normal(0, 8, n))   # price in lakhs
    df = pd.DataFrame({"Area": area, "Bedrooms": bedrooms, "Bathrooms": bathrooms,
        "Age": age,
                       "Distance_City": distance, "Garage": garage, "Price":
                           price.round(2)})
    df.to_csv(CSV_PATH, index=False)
    return df

df = load_data()

# ---------- 2. Data Preprocessing ----------
print("Missing values before cleaning:", int(df.isnull().sum().sum()))
df = df.fillna(df.median(numeric_only=True))       # handle missing values
df = df.drop_duplicates().reset_index(drop=True)   # remove duplicate rows

# ---------- 3. Feature Selection ----------
FEATURES = ["Area", "Bedrooms", "Bathrooms", "Age", "Distance_City", "Garage"]
X, y = df[FEATURES], df["Price"]

# ---------- 4. Train-Test Split (80-20) ----------
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)

# ---------- 5. Model Training ----------
model = LinearRegression()
model.fit(X_train, y_train)

# ---------- 6. Prediction and Evaluation ----------
pred = model.predict(X_test)
mse = mean_squared_error(y_test, pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, pred)
r2 = r2_score(y_test, pred)

print("Total records      :", len(df))
print("Training records   :", len(X_train))
print("Testing records    :", len(X_test))
print("Intercept          :", round(model.intercept_, 4))
for name, coef in zip(FEATURES, model.coef_):
    print(f"Coefficient {name:<13}: {coef:.4f}")
print(f"MAE                : {mae:.4f} lakhs")
print(f"MSE                : {mse:.4f}")
print(f"RMSE               : {rmse:.4f} lakhs")
print(f"R2 Score           : {r2:.4f}")

# ---------- 7. Visualization ----------
plt.figure(figsize=(6.5, 5))
plt.scatter(y_test, pred, alpha=0.5, color="#2e7d32", edgecolors="none")
lims = [min(y_test.min(), pred.min()), max(y_test.max(), pred.max())]
plt.plot(lims, lims, color="#d32f2f", linestyle="--", label="Perfect prediction")
plt.title("House Price: Actual vs Predicted (Test Set)")
plt.xlabel("Actual Price (lakhs)")
plt.ylabel("Predicted Price (lakhs)")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("actual_vs_predicted.png", dpi=200)

# ---------- 8. Save model for the web app ----------
joblib.dump(model, "model.pkl")
json.dump({"mae": mae, "mse": mse, "rmse": rmse, "r2": r2, "train": len(X_train),
    "test": len(X_test)},
          open("metrics.json", "w"))
```

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>House Price Predictor</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
      rel="stylesheet">
  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
<div class="container" style="max-width:520px; margin-top:40px">
  <div class="card-form">
    <h2 class="brand">House Price Predictor</h2>
    <h5>Welcome, {{ username }}!</h5>
    <form action="{{ url_for('dashboard') }}" method="post" class="mt-3">
      <input type="number" min="100" name="Area" value="{{ request.form.get('Area', '')
          }}"
             placeholder="Area (sq ft)" class="form-control mb-3" required>
      <select name="Bedrooms" class="form-select mb-3" required>
        <option value="">Bedrooms</option>
        {% for n in range(1, 7) %}
        <option value="{{ n }}" {{ 'selected' if request.form.get('Bedrooms') ==
            n|string }}>{{ n }}</option>
        {% endfor %}
      </select>
      <select name="Bathrooms" class="form-select mb-3" required>
        <option value="">Bathrooms</option>
        {% for n in range(1, 6) %}
        <option value="{{ n }}" {{ 'selected' if request.form.get('Bathrooms') ==
            n|string }}>{{ n }}</option>
        {% endfor %}
      </select>
      <input type="number" min="0" name="Age" value="{{ request.form.get('Age', '') }}"
             placeholder="Age of the house (years)" class="form-control mb-3" required>
      <input type="number" step="0.1" min="0" name="Distance_City" value="{{
          request.form.get('Distance_City', '') }}"
             placeholder="Distance from city centre (km)" class="form-control mb-3"
                 required>
      <select name="Garage" class="form-select mb-3" required>
        <option value="">Garage / Parking</option>
        <option value="1" {{ 'selected' if request.form.get('Garage') == '1'
            }}>Yes</option>
        <option value="0" {{ 'selected' if request.form.get('Garage') == '0'
            }}>No</option>
      </select>
      <button type="submit" class="btn btn-power w-100">Predict</button>
    </form>
    {% if prediction_text %}
      <div class="result-box">{{ prediction_text }}</div>
    {% endif %}
    <a href="{{ url_for('logout') }}" class="d-block text-center mt-3">Logout</a>
  </div>
</div>
<div class="footer-dark">House Price Predictor &copy; 2025</div>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.j
    s"></script>
</body>
</html>
```

Home.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>House Price Prediction</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
      rel="stylesheet">
```

```
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
      awesome/6.4.2/css/all.min.css">
  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
<div class="container">
  <header class="d-flex flex-wrap justify-content-between align-items-center py-3 mb-4
      border-bottom">
    <span class="fs-3 brand"><i class="fas fa-house"></i> House Price Prediction</span>
    <ul class="nav nav-pills">
      <li class="nav-item"><a href="{{ url_for('home') }}" class="nav-link active"
          style="background:#1b5e20;color:#fff;border-radius:10px">Home</a></li>
      <li class="nav-item"><a href="#features" class="nav-link">Features</a></li>
      <li class="nav-item"><a href="#about" class="nav-link">About</a></li>
      <li class="nav-item"><a href="{{ url_for('register') }}" class="nav-
          link">Register</a></li>
      <li class="nav-item"><a href="{{ url_for('login') }}" class="nav-
          link">Login</a></li>
      <li class="nav-item"><a href="{{ url_for('login') }}" class="btn btn-power
          ms-2">Get Started</a></li>
    </ul>
  </header>
</div>

<div class="container py-4">
  <div class="row align-items-center g-5">
    <div class="col-lg-6">
      <h1 class="hero-title">House Price Prediction</h1>
      <p class="lead">Estimate the market price of a house using a machine learning
          model trained on
        area, rooms, age, location and parking details.</p>
      <a href="{{ url_for('login') }}" class="btn btn-lg btn-power px-4">Get Started</a>
    </div>
    <div class="col-lg-6 text-center"><i class="fas fa-house-chimney" style="font-
        size:12rem;color:#66bb6a;filter:drop-shadow(0 10px 18px
        rgba(27,94,32,.35))"></i></div>
  </div>
</div>

<div class="container py-5" id="features">
  <h2 class="text-center brand mb-4">Features</h2>
  <div class="row g-4 text-center feature">
    <div class="col-md-4"><div class="feature-card"><i class="fas fa-chart-
        line"></i><h4>Accurate Forecasts</h4>
      <p>Linear Regression learns how each property feature adds to or reduces the
          price.</p></div></div>
    <div class="col-md-4"><div class="feature-card"><i class="fas fa-ruler-
        combined"></i><h4>Property Aware</h4>
      <p>Area, bedrooms, bathrooms, age, distance and parking are used as price
          drivers.</p></div></div>
    <div class="col-md-4"><div class="feature-card"><i class="fas fa-user-
        check"></i><h4>Secure and Simple</h4>
      <p>Login-protected dashboard with a simple form for instant
          predictions.</p></div></div>
  </div>
</div>

<div class="container py-5" id="about">
  <h2 class="text-center brand mb-4">About Us</h2>
  <p class="text-center mx-auto" style="max-width:720px">House Price Prediction is a
      tool that
    uses past property data to estimate what a house is worth. Accurate estimates help
        buyers, sellers
    and agents set fair prices and make informed decisions.</p>
</div>

<div class="footer-dark">&copy; 2025 House Price Prediction</div>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.j
    s"></script>
</body>
</html>
```

Login.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Login</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
      rel="stylesheet">
  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body class="d-flex align-items-center justify-content-center" style="min-height:100vh">
<div class="card-form" style="width:420px">
  <h2 class="text-center brand mb-4">Login</h2>
  {% with messages = get_flashed_messages(with_categories=true) %}
    {% for category, message in messages %}
      <div class="alert alert-{{ 'danger' if category == 'error' else 'success' }}">{{
          message }}</div>
    {% endfor %}
  {% endwith %}
  <form action="{{ url_for('login') }}" method="post">
    <div class="mb-3">
      <label for="username" class="form-label">Username</label>
      <input type="text" class="form-control" name="username" id="username"
             placeholder="Enter your username" required>
    </div>
    <div class="mb-3">
      <label for="password" class="form-label">Password</label>
      <input type="password" class="form-control" name="password" id="password"
             placeholder="Enter your password" required>
    </div>
    <div class="d-grid"><button type="submit" class="btn btn-power">Login</button></div>
    <p class="text-center mt-3">Don't have an account? <a href="{{ url_for('register')
        }}">Register</a></p>
  </form>
</div>
</body>
</html>
```

Register.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Register</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
      rel="stylesheet">
  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body class="d-flex align-items-center justify-content-center" style="min-height:100vh">
<div class="card-form" style="width:420px">
  <h2 class="text-center brand mb-4">Register</h2>
  {% with messages = get_flashed_messages(with_categories=true) %}
    {% for category, message in messages %}
      <div class="alert alert-{{ 'danger' if category == 'error' else 'success' }}">{{
          message }}</div>
    {% endfor %}
  {% endwith %}
  <form action="{{ url_for('register') }}" method="post">
    <div class="mb-3">
      <label for="name" class="form-label">Name</label>
      <input type="text" class="form-control" name="username" id="name"
          placeholder="Enter your name" required>
    </div>
    <div class="mb-3">
      <label for="gender" class="form-label">Gender</label>
      <select class="form-select" name="gender" id="gender" required>
        <option value="">Select Gender</option>
        <option value="Male">Male</option>
```

```
        <option value="Female">Female</option>
        <option value="Other">Other</option>
      </select>
    </div>
    <div class="mb-3">
      <label for="age" class="form-label">Age</label>
      <input type="number" class="form-control" name="age" id="age" placeholder="Enter
          your age" required>
    </div>
    <div class="mb-3">
      <label for="password" class="form-label">Password</label>
      <input type="password" class="form-control" name="password" id="password"
             placeholder="Enter password" required>
    </div>
    <div class="d-grid"><button type="submit" class="btn btn-
        power">Register</button></div>
    <p class="text-center mt-3">Already have an account? <a href="{{ url_for('login')
        }}">Sign In</a></p>
  </form>
</div>
</body>
</html>
```

Style.css

```
body { font-family: 'Segoe UI', Arial, sans-serif; color: #1f2937;
       background: linear-gradient(135deg, #e8f5e9 0%, #c8e6c9 100%); min-height: 100vh;
             }
.brand { color: #1b5e20; font-weight: 700; }
.btn-power { background: linear-gradient(90deg, #66bb6a, #2e7d32); color: #fff; font-
    weight: 600;
             border: none; border-radius: 10px; padding: 10px 20px;
             box-shadow: 0 4px 12px rgba(46, 125, 50, 0.35); }
.btn-power:hover { color: #fff; filter: brightness(1.08); }
.card-form { background: #ffffff; border-radius: 18px; padding: 36px;
             box-shadow: 0 12px 35px rgba(27, 94, 32, 0.18); border-top: 6px solid
                 #66bb6a; }
.form-control, .form-select { border-radius: 10px; padding: 10px 14px; border: 1px solid
     #e5e7eb; }
.form-control:focus, .form-select:focus { border-color: #66bb6a; box-shadow: 0 0 0 3px
    rgba(102, 187, 106, 0.25); }
.nav-link { color: #1b5e20; font-weight: 500; }
.hero-title { font-size: 2.8rem; font-weight: 800; line-height: 1.15;
              background: linear-gradient(90deg, #1b5e20, #66bb6a); -webkit-background-
                  clip: text;
              background-clip: text; color: transparent; }
.feature-card { background: #fff; border-radius: 16px; padding: 28px 20px; height: 100%;
                box-shadow: 0 8px 24px rgba(0, 0, 0, 0.08); }
.feature i { font-size: 2.2rem; color: #2e7d32; margin-bottom: 10px; }
.result-box { background: #e8f5e9; border-left: 5px solid #2e7d32; border-radius: 8px;
              padding: 14px; margin-top: 18px; font-weight: 600; color: #1b5e20; text-
                  align: center; }
.footer-dark { background: linear-gradient(90deg, #1b5e20, #66bb6a); color: #fff;
               text-align: center; padding: 16px; margin-top: 40px; }
```

App.py

```
import joblib
import pandas as pd
from flask import Flask, render_template, request, redirect, url_for, flash
from flask_sqlalchemy import SQLAlchemy
from flask_login import (LoginManager, UserMixin, login_user,
                         login_required, logout_user, current_user)
from werkzeug.security import generate_password_hash, check_password_hash

app = Flask(__name__)
app.config['SECRET_KEY'] = 'change-this-secret-key'
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///site.db'
db = SQLAlchemy(app)

login_manager = LoginManager(app)
```

```
login_manager.login_view = 'login'

model = joblib.load('model.pkl')          # trained Linear Regression model
FEATURES = ['Area', 'Bedrooms', 'Bathrooms', 'Age', 'Distance_City', 'Garage']

# --- Models ---
class User(UserMixin, db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(150), unique=True, nullable=False)
    gender = db.Column(db.String(20))
    age = db.Column(db.Integer)
    password = db.Column(db.String(200), nullable=False)

@login_manager.user_loader
def load_user(user_id):
    return db.session.get(User, int(user_id))

# --- Routes ---
@app.route('/')
def home():
    return render_template('Home.html')

@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        username = request.form['username']
        if User.query.filter_by(username=username).first():
            flash('User already exists', 'error')
            return redirect(url_for('register'))
        user = User(username=username,
                    gender=request.form['gender'],
                    age=int(request.form['age']),
                    password=generate_password_hash(request.form['password']))
        db.session.add(user)
        db.session.commit()
        flash('Registered successfully', 'success')
        return redirect(url_for('login'))
    return render_template('Register.html')

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        user = User.query.filter_by(username=request.form['username']).first()
        if user and check_password_hash(user.password, request.form['password']):
            login_user(user)
            return redirect(url_for('dashboard'))
        flash('Invalid credentials', 'error')
    return render_template('Login.html')

@app.route('/dashboard', methods=['GET', 'POST'])
@login_required
def dashboard():
    prediction_text = ""
    if request.method == 'POST':
        values = [[float(request.form['Area']),
                   int(request.form['Bedrooms']),
                   int(request.form['Bathrooms']),
                   float(request.form['Age']),
                   float(request.form['Distance_City']),
                   int(request.form['Garage'])]]
        row = pd.DataFrame(values, columns=FEATURES)
        predicted = max(0, model.predict(row)[0])
        prediction_text = f"Predicted House Price: Rs. {predicted:.2f} Lakhs"
    return render_template('Index.html', username=current_user.username,
                           prediction_text=prediction_text)

@app.route('/logout')
@login_required
def logout():
    logout_user()
    return redirect(url_for('login'))
```

```
with app.app_context():
    db.create_all()

if __name__ == '__main__':
    app.run(debug=True)
```

OUTPUT:

Model training and evaluation (console output)

```
Missing values before cleaning: 0
Total records      : 1500
Training records   : 1200
Testing records    : 300
Intercept          : 8.9066
Coefficient Area         : 0.0446
Coefficient Bedrooms     : 4.6760
Coefficient Bathrooms    : 2.4117
Coefficient Age          : -0.5671
Coefficient Distance_City: -1.1613
Coefficient Garage       : 5.2535
MAE                : 6.7774 lakhs
MSE                : 72.2298
RMSE               : 8.4988 lakhs
R2 Score           : 0.9700
```

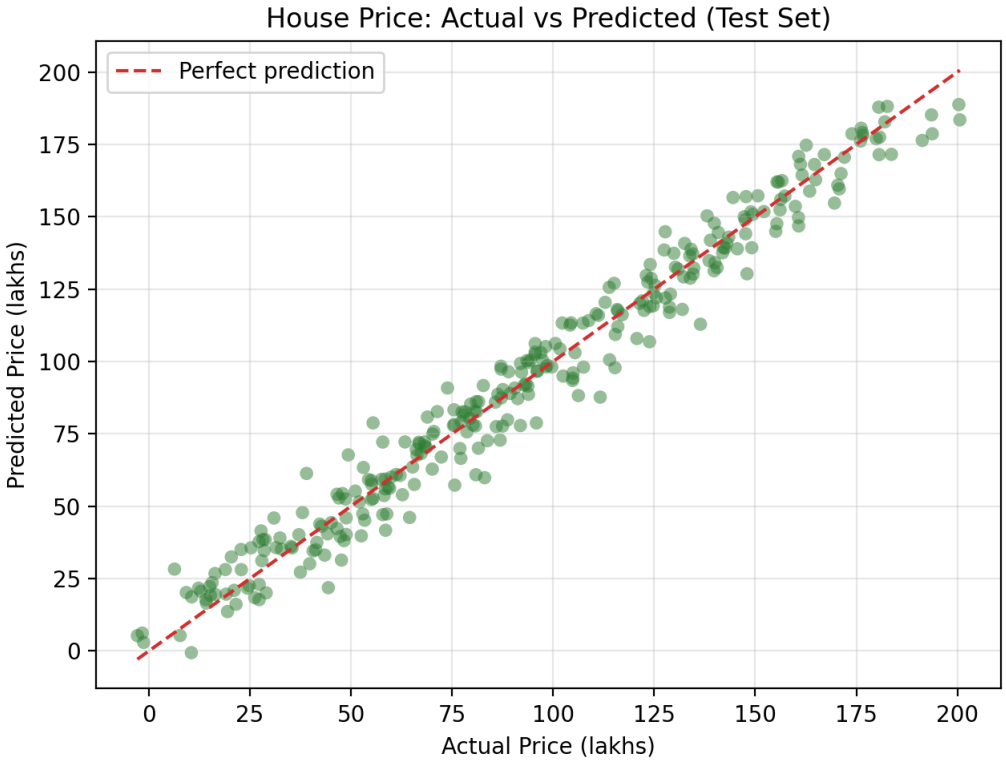


Fig. 1: Actual vs predicted house prices on the test set

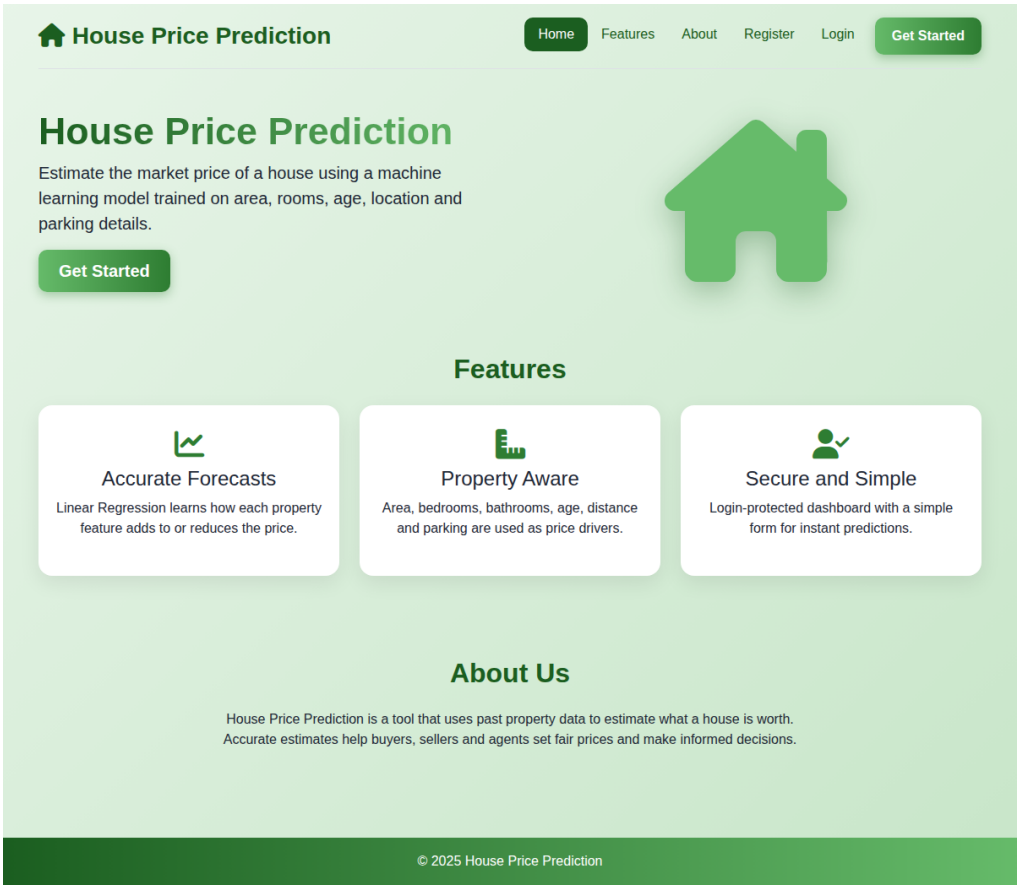


Fig. 2: Home page

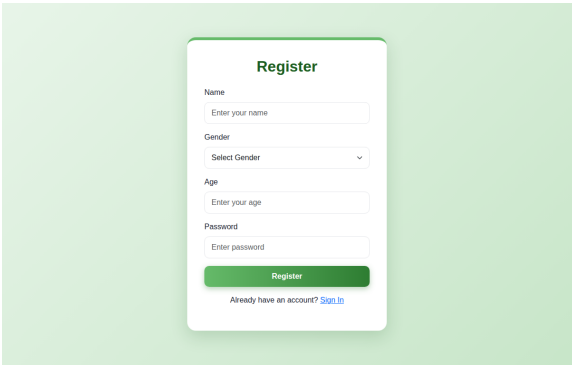


Fig. 3: Register page

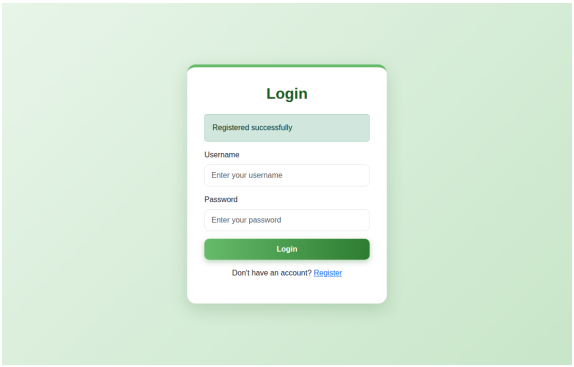


Fig. 4: Login page

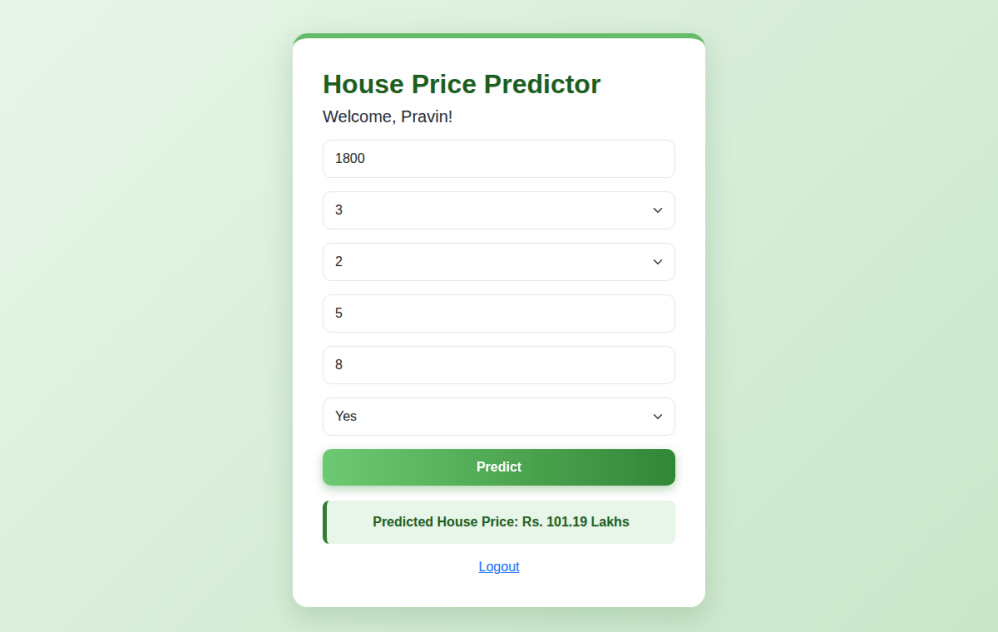


Fig. 5: Dashboard with prediction result

RESULT:

The Linear Regression model was successfully trained on 1200 records and tested on 300 records to predict house prices. It achieved an R^2 score of 0.9700, an RMSE of 8.50 lakhs and an MAE of 6.78 lakhs on the test set. The predicted prices lay close to the actual prices, showing that the model effectively captured how area, rooms, age, distance and parking affect the price. The trained model was deployed in a Flask web application that returns the predicted price for user-entered property details.